

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica



ELABORATO DI ARCHITETTURA DEI SISTEMI INTEGRATI

Prof. Petra Nicola

a.a. 2023-24

Studenti:

Aurelio Salvati M63/1619

Indice

Rete Neurale	3
Neurone	3
Carry Save Adder	5
Array Multiplier Carry-Save	7
Implementazione	10
Simulazione	11
Architettura 1	12
Implementazione	14
Simulazione	16
Architettura 2	17
Implementazione	18
Simulazione	21
 Datapath Aritmetico	 22
Progetto e Architettura	23
Datapath Pipelined	23
Implementazione	25
Rippe Carry Adder	25
Buffer	26
Datapath	27
Simulazione	29
Generaione casuale input	29
Lettura da file	29
Riconoscimento codice operazione	29
Test Bench	30
 Appendice	 33
Multiplexer	33
Registro	33

Rete Neurale

The purpose of this exerciser is to implement a simple accelerator for convolutional neural network according to the following architecture. The inputs of the network are 4 integer values X1, X2, X3 and X4, the computed result is the integer value Y. The accelerator contains an arithmetic datapath and a finite state machine (FSM). The arithmetic datapath contains only registers and neurons while the FSM controls the execution of the arithmetic datapath and the communication of the accelerator with the other circuits through the signals valid, ready, valid_out and ready_out. In order to complete the project, we will first design the neurons and arrange them in the arithmetic datapath. We will then define the operations that the FSM must perform and will set the protocol used for the communication of the accelerator with the other circuits.

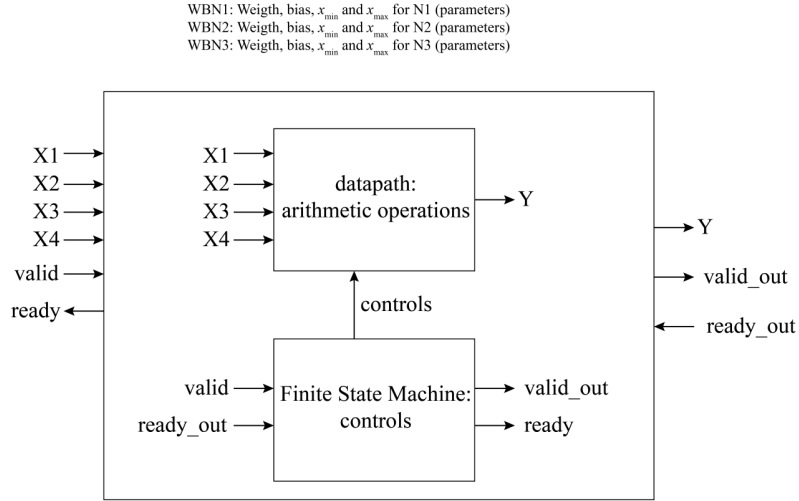


Figure 1: Acceleratore Rete Neurale

Neurone

Il neurone esegue operazioni aritmetiche secondo le seguenti equazioni:

$$\text{Somma} = X1 \cdot W1 + X2 \cdot W2 + X3 \cdot W3 + X4 \cdot W4 + \text{bias}$$

dove Somma rappresenta la somma pesata dei 4 input dei neuroni. I pesi ($W1, W2, W3, W4$) e il valore di bias vengono forniti al neurone come input.

L'uscita del neurone (uscita) viene calcolata come il risultato di una funzione di attivazione (f) applicata al risultato della somma ponderata:

$$\text{uscita} = f(\text{Somma})$$

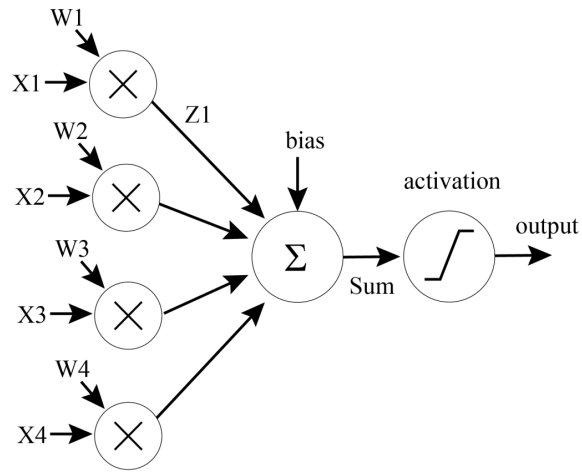


Figure 2: Neurone

La funzione di attivazione del neurone gestisce l'ingresso x , un segnale intero a 12 bit (risultato della somma ponderata). L'output della funzione di attivazione è definito come segue:

$$\text{Uscita} = \begin{cases} x & \text{se } x \geq \text{xmin e } x \leq \text{xmax} \\ \text{xmin} & \text{se } x < \text{xmin} \\ \text{xmax} & \text{se } x > \text{xmax} \end{cases}$$

Dove xmin e xmax sono valori interi a 12 bit forniti come input per la funzione di attivazione. Il risultato finale della funzione di attivazione sarà un valore intero a 8 bit.

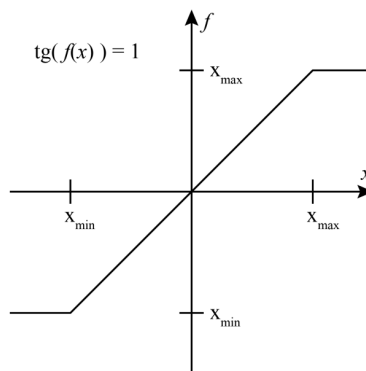


Figure 3: Funzione di attivazione

Carry Save Adder

Per implementare l'addizionatore all'interno del neurone, considerando la molteplicità di operandi da sommare (5), è stata scelta la realizzazione di un carry-save adder. In questo approccio, vengono sommati contemporaneamente tutti i bit di peso omologo di A, B, C, D, E, ottenendo un bit di somma con lo stesso peso e un bit di riporto di peso maggiore.

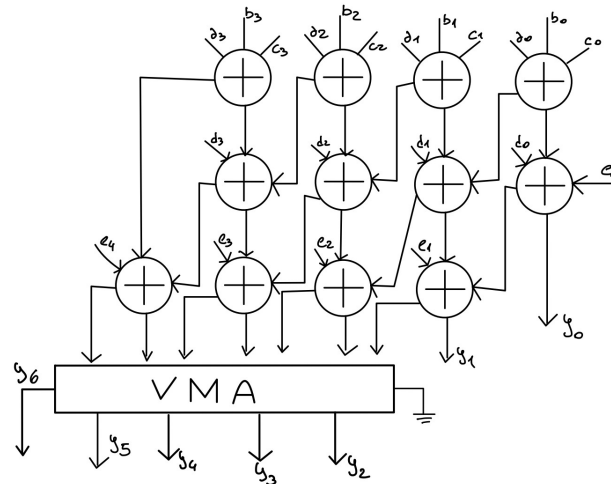


Figure 4: Carry Save 5 operandi a 3 bit

È importante notare che il bit di riporto non si propaga, ma viene sommato con altri bit di peso omologo di E oppure con il risultato delle somme di A, B, C, D. Per ottenere la somma complessiva, è necessario utilizzare un addizionatore finale noto come Vector Merging Adder (VMA), che risulta essere un Ripple carry adder.

Implementazione

```
module Carry_Save #(parameter N=4) (
    input wire [N-1:0] A, B, C, D, E,
    output wire [N+2:0] Sum,
    output wire Cout
);

    wire [N+2:0] A_ext, B_ext, C_ext, D_ext, E_ext;

    assign A_ext={A[N-1],A[N-1],A[N-1],A};
    assign B_ext={B[N-1],B[N-1],B[N-1],B};
    assign C_ext={C[N-1],C[N-1],C[N-1],C};
    assign D_ext={D[N-1],D[N-1],D[N-1],D};
    assign E_ext={E[N-1],E[N-1],E[N-1],E};

    wire [N+2:0] Sum_temp1, Carry_temp1, Carry_temp2;
```

```

wire [N+2:1] Sum_temp2;
wire [N+3:2] Sum_temp3;
wire [N+3:1] Carry_temp3;

// Full Adder prima fila
genvar i;
generate
  for (i = 0; i < N+3; i = i + 1) begin : gen_adder_1
    FullAdder FA (
      .A(A_ext[i]),
      .B(B_ext[i]),
      .Cin(C_ext[i]),
      .Sum(Sum_temp1[i]),
      .Cout(Carry_temp1[i])
    );
  end

// primo FA seconda fila
FullAdder FA_0_2 (
  .A(Sum_temp1[0]),
  .B(D[0]),
  .Cin(E[0]),
  .Sum(Sum[0]),
  .Cout(Carry_temp2[0])
);

// seconda fila
for (i = 1; i < N+3; i = i + 1) begin : gen_adder_2
  FullAdder FA (
    .A(Sum_temp1[i]),
    .B(D_ext[i]),
    .Cin(Carry_temp1[i-1]),
    .Sum(Sum_temp2[i]),
    .Cout(Carry_temp2[i])
  );
end

// primo FA terza fila
FullAdder FA_0_3 (
  .A(Sum_temp2[1]),
  .B(E[1]),
  .Cin(Carry_temp2[0]),
  .Sum(Sum[1]),
  .Cout(Carry_temp3[1])
);

// terza fila
for (i = 2; i < N+3; i = i + 1) begin : gen_adder_3
  FullAdder FA (
    .A(Sum_temp2[i]),

```

```

        .B(E_ext[i]),
        .Cin(Carry_temp2[i-1]),
        .Sum(Sum_temp3[i]),
        .Cout(Carry_temp3[i])
    );
end

// ultimo FA merging cout
FullAdder FA_3_N (
    .A( Carry_temp1[N+2]),
    .B(Carry_temp2[N+2]),
    .Cin(1'b0),
    .Sum(Sum_temp3[N+3]),
    .Cout(Carry_temp3[N+3])
);

endgenerate

RippleCarryAdder#(N+3) VMA(.A({1'b0, Sum_temp3}), .B(Carry_temp3),
    .Cin(1'b0), .Sum(Sum[N+2:2]), .Cout(Cout));

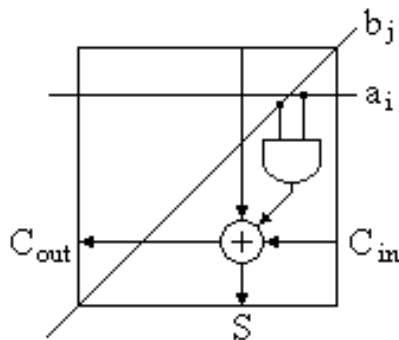
endmodule

```

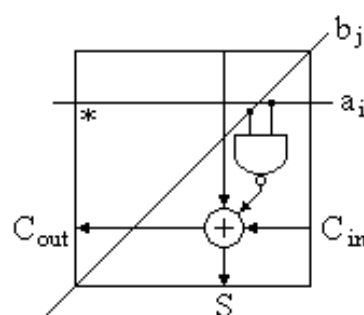
Nota: Per effettuare la somma di numeri rappresentati in complemento a due alla base, è stato necessario estendere il segno per tutti gli operandi, portando il numero di bit a un valore pari ai bit inclusi nel risultato finale. Questo assicura che i numeri siano rappresentati correttamente in base alla dimensione del risultato desiderato.

Array Multiplier Carry-Save

Per la moltiplicazione tra due segnali espressi in complemento alla base, si è scelto di utilizzare una struttura Carry-Save, in cui vengono eseguite operazioni adeguate per gestire il segno degli operandi e della somma. Le due celle base di questa struttura sono le seguenti:



(a) Cella AND



(b) Cella NAND

Nel particolare, la cella NAND viene adoperata per la gestione del segno, poiché la moltiplicazione logica con l'MSB di ciascuno degli operandi va sottratta al risultato finale. Per fare ciò, i risultati del prodotto parziale vengono complementati e viene aggiunto un termine costante per concludere il complemento.

$$\begin{array}{ccccccc}
 & & & & \text{estensione del segno} & & \\
 & & & & \downarrow & & \\
 0 & a_1b_2 & a_0b_2 & 0 & 0 & \rightarrow & 1 & 1 & \overline{a_1b_2} & \overline{a_0b_2} & 1 & 1 & + \\
 & & & & & & 0 & 0 & 0 & 0 & 0 & 1 & + \\
 \\
 & & & & & & 1 & 1 & \overline{a_2b_1} & \overline{a_2b_0} & 1 & 1 & + \\
 0 & a_2b_1 & a_2b_0 & 0 & 0 & \rightarrow & 0 & 0 & 0 & 0 & 0 & 1 & +
 \end{array}$$

Figure 6: Gestione del Segno per operandi in complemento alla base

Di seguito la struttura finale:

(Le celle con l'asterisco hanno una **NAND** per calcolare i prodotti parziali).

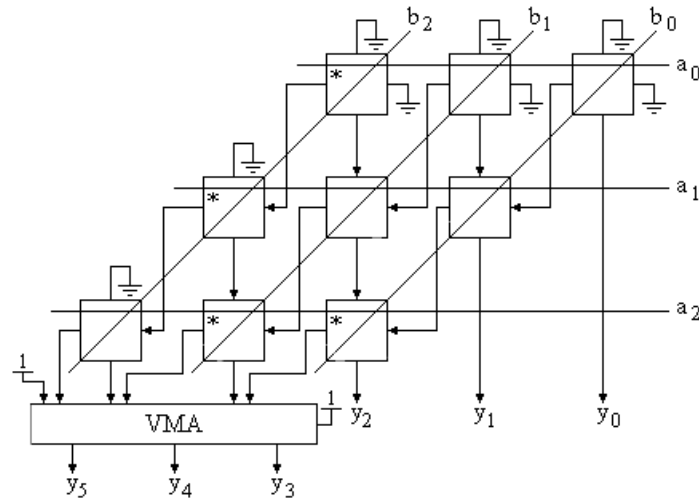


Figure 7: Carry Save Multiplier a 3 bit

Implementazione

```

module array_multiplier
#(parameter N=3)
(
    input wire [N-1:0] A, B,
    output wire [2*N-1:0] Prod,
    output wire Cout
);

    wire [2*N-2:0] mac_out [N-2:0];
    wire [2*N-2:0] carry_out [N-2:0];
    wire [N-1:0] last_sum ,last_cout;

```



```

    //MAC prima fila
    genvar i,j;
    generate
        for (i = 0; i < N-1; i = i + 1) begin : gen_adder_1
            MAC_Cell MC (
                .A(A[0]),
                .B(B[i]),
                .C(1'b0),
                .Cin(1'b0),
                .Sum_out(mac_out[0][i]),
                .Cout_out(carry_out[0][i])
            );
        end
        MAC_nand MC (
            .A(A[0]),
            .B(B[N-1]),
            .C(1'b0),
            .Cin(1'b0),
            .Sum_out(mac_out[0][N-1]),
            .Cout_out(carry_out[0][N-1])
        );
        assign Prod[0]= mac_out[0][0];
    endgenerate

    // MAC i-esima fila
    generate
        for(j=1; j<N-1;j=j+1) begin: matrix
            for (i = j; i < N+j-1; i = i + 1) begin : gen_adder_i
                MAC_Cell MC (
                    .A(A[j]),
                    .B(B[i-j]),
                    .C(mac_out[j-1][i]),
                    .Cin(carry_out[j-1][i-1]),
                    .Sum_out(mac_out[j][i]),
                    .Cout_out(carry_out[j][i])
                );
            end
            MAC_nand MC (
                .A(A[j]),
                .B(B[N-1]),
                .C(1'b0),
                .Cin(carry_out[j-1][N-2+j]),
                .Sum_out(mac_out[j][N-1+j]),
                .Cout_out(carry_out[j][N-1+j])
            );

            assign Prod[j]= mac_out[j][j];
        end
    endgenerate

```

```

endgenerate

// MAC ultima fila
generate
for (i = N-1; i < 2*N-2; i = i + 1) begin : gen_adder_last
    MAC_nand MC (
        .A(A[N-1]),
        .B(B[i-N+1]),
        .C(mac_out[N-2][i]),
        .Cin(carry_out[N-2][i-1]),
        .Sum_out(last_sum[i-N+1]),
        .Cout_out(last_cout[i-N+1])
    );
end
MAC_Cell MC_last (
    .A(A[N-1]),
    .B(B[N-1]),
    .C(1'b0),
    .Cin(carry_out[N-2][2*N-3]),
    .Sum_out(last_sum[N-1]),
    .Cout_out(last_cout[N-1])
);
assign Prod[N-1] = last_sum[0];
endgenerate

// Ripple Carry Adder
RippleCarryAdder #(N) VMA (
    .A({1'b1, last_sum[N-1:1]}),
    .B(last_cout),
    .Cin(1'b1),
    .Sum(Prod[2*N-1:N]),
    .Cout(Cout)
);

endmodule

```

Implementazione

Funzione di attivazione:

```

module activationFunc(x,f,xmin,xmax);

input signed [11:0] x;
output signed [7:0] f;
input signed [11:0] xmin, xmax;

assign f = (x < xmin) ? xmin : (x > xmax) ? xmax : x;

endmodule

```

Neurone:

```
module neuron(X1, X2, X3, X4, W1, W2, W3, W4, bias, xmin, xmax, Y);
    input signed [7:0] X1, X2, X3, X4, W1, W2, W3, W4;
    input signed [15:0] bias;
    input signed [11:0] xmin, xmax;
    output signed [7:0] Y;

    wire signed [15:0] Z1,Z2,Z3,Z4;
    wire signed [18:0] Sum;

    array_multiplier #(8) multiplier_1(X1,W1,Z1);
    array_multiplier #(8) multiplier_2(X2,W2,Z2);
    array_multiplier #(8) multiplier_3(X3,W3,Z3);
    array_multiplier #(8) multiplier_4(X4,W4,Z4);

    Carry_Save #(16) adder(Z1,Z2,Z3,Z4,bias,Sum[18:0]);
    activationFunc attivazione(Sum[18:7],Y,xmin,xmax);

endmodule
```

Simulazione

È presentato di seguito l'output del test bench fornito, il quale attesta il successo di tutti e 100 i test a cui è stato sottoposto l'acceleratore neurale.

```
100 cases evaluated,          0 errors found
```

Architettura 1

L'architettura proposta è una struttura pipelined che comprende tre neuroni e tre registri. Le uscite dei primi due neuroni vengono memorizzate in due registri separati, i quali alimentano l'ultimo neurone per ottenere il risultato finale.

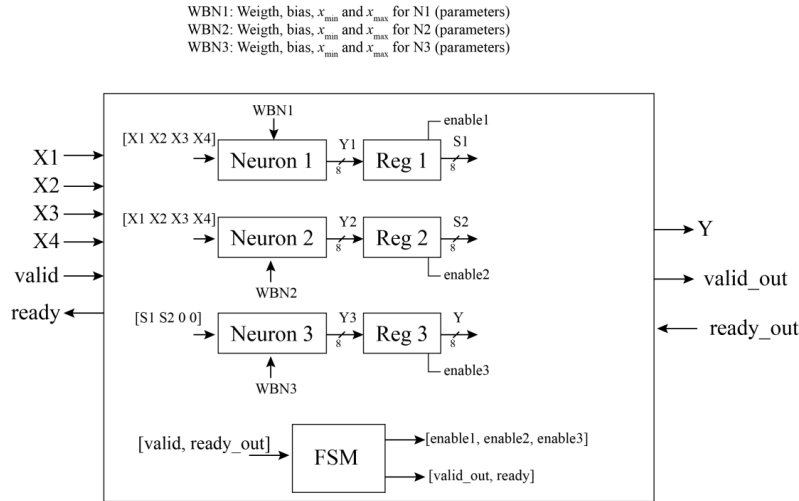


Figure 8: Architettura 1

I registri sono sensibili al fronte di salita di un segnale di clock. I parametri per ciascun neurone, come weight, bias, $x_{\min}$ e $x_{\max}$, sono dichiarati come parametri all'interno del modulo acceleratore. L'FSM abilita o disabilita i registri in base alla disponibilità dei dati di input. Il segnale di abilitazione deve essere alto quando il registro può essere aggiornato, altrimenti basso. L'FSM riceve un segnale di clock e cambia il suo stato sul fronte di salita del clock.

Il protocollo di comunicazione, basato sui segnali di handshaking **ready**, **valid**, **ready_out** e **valid_out**, è definito nel seguente modo:

- Si presuppone che un circuito denominato "feeder" fornisca all'acceleratore i 4 ingressi **X1**, **X2**, **X3** e **X4**, mentre un circuito denominato "consumer" utilizzerà il risultato calcolato dall'acceleratore.
- I segnali **valid** e **ready** vengono utilizzati dall'acceleratore per comunicare con l'alimentatore, mentre **valid_out** e **ready_out** vengono utilizzati per comunicare con il consumatore.
- Se **valid** e **ready** sono alti quando si verifica il fronte di salita del clock, i valori di **X1**, **X2**, **X3** e **X4** vengono elaborati dall'acceleratore, e i valori dei registri **Reg 1** e **Reg 2** possono essere aggiornati con i risultati parziali forniti da **Neuron 1** e **Neuron 2**. L'alimentatore presume che i dati siano stati "consumati" dall'acceleratore e non manterrà il loro valore all'ingresso del circuito sul successivo fronte di salita dell'orologio.

- Se **valid** è basso, i valori $X1-X4$ non contengono dati validi e non devono essere elaborati dall'acceleratore.
- Se **valid** è alto e **ready** è basso, i valori $X1-X4$ contengono dati validi, ma l'acceleratore non è in grado di elaborarli. L'alimentatore manterrà i segnali $X1-X4$ e manterrà alto il segnale **valid** finché l'acceleratore non sarà pronto ad elaborarli.

Il protocollo implementato dai segnali **ready_out** e **valid_out** è simile a quello che coinvolge **valid** e **ready**. In questo caso, **valid_out** viene affermato (diventa alto) quando l'acceleratore ha dati validi nel registro **Reg 3** e può fornire il risultato al consumatore che sta gestendo il segnale **ready_out**. I valori in **Reg 3** devono essere mantenuti finché il consumatore non è pronto a leggerli. I valori possono essere considerati "letti" se **valid_out** e **ready_out** sono entrambi alti sul fronte di salita dell'orologio.

L'automa a stati finiti che guida l'architettura proposta è il seguente:

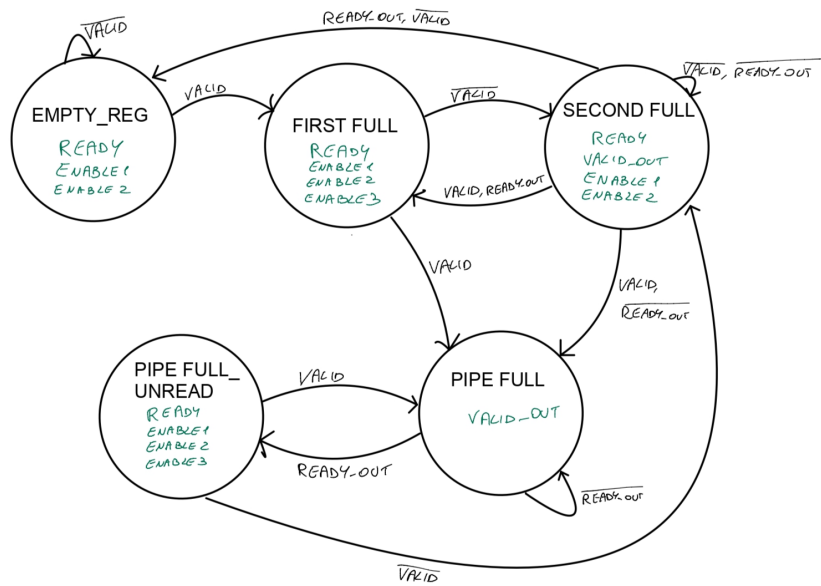


Figure 9: Automa a Stati Finiti

State	Description
Empty registers	The circuit has no valid data in the internal registers (Reg 1, Reg 2 and Reg 3). The signal ready is high because the accelerator can receive new data. The signal valid_out is low because there is no valid data in Reg 3.
First pipe stage full, second pipe stage empty	Reg 1 and Reg 2 contain data computed by valid inputs. Reg 3 does not contain valid data. The output computed by the Neuron 3 can be stored in Reg 3 on the next clock rising edge. The signal ready is high while the valid_out signal is low.
First pipe stage empty, second pipe stage full	Reg 1 and Reg 2 do not contain valid data while Reg 3 contains a computed valid result. Ready and valid_out are both high.
Pipe full, result unread	Reg 1, Reg 2 and Reg 3 contain valid data. The result in Reg 3 is ready to be read and it cannot be overwritten until the consumer reads its value. Ready is low until the output is read.
Pipe full, result read	Reg 1, Reg 2 and Reg 3 contain valid data. The result in Reg 3 has been read by the consumer and its content can be overwritten with the values computed by neuron 3. Ready is high and a new batch of inputs can enter the pipeline.

Figure 10: Tabella FSM

Implementazione

Descrizione dell'automa a stati finiti:

```

module fsm(ready, valid, ready_out, valid_out, clk, arst, en);
    input valid, ready_out;
    output ready, valid_out;
    input clk, arst;
    output [2:0] en;

    wire clk, arst, valid, ready_out;
    reg [2:0] en;
    reg ready=0, valid_out=0;
    reg [2:0] ac_state=3'b000;
    parameter [2:0] EMPTY=3'b000, FIRST=3'b001, SECOND=3'b010,
    PIPE_UNREAD=3'b011, PIPE_READ=3'b100;

    always @(posedge arst or posedge clk)
    begin
        if (arst)
            ac_state <= EMPTY;
        else
            case (ac_state)
                EMPTY: ac_state <= (valid) ? FIRST : EMPTY;
                FIRST: ac_state <= (valid) ? PIPE_UNREAD : SECOND;
                SECOND: ac_state <= (valid) ?
                    (ready_out ? FIRST : PIPE_UNREAD) :
                    (ready_out ? EMPTY : SECOND);
                PIPE_UNREAD: ac_state <= (ready_out) ? PIPE_READ : PIPE_UNREAD;
                PIPE_READ: ac_state <= (valid) ? PIPE_UNREAD : SECOND;
                default: ac_state <= EMPTY;
            endcase
    end
end

```

```

always @(ac_state)
begin
    case (ac_state)
        EMPTY: begin
            ready <= 1;
            valid_out <= 0;
            en = 3'b011;
        end
        FIRST: begin
            ready <= 1;
            valid_out <= 0;
            en = 3'b111;
        end
        SECOND: begin
            ready <= 1;
            valid_out <= 1;
            en = 3'b011;
        end
        PIPE_UNREAD: begin
            ready <= 0;
            valid_out <= 1;
            en = 3'b000;
        end
        PIPE_READ: begin
            ready <= 1;
            valid_out <= 0;
            en = 3'b111;
        end
    endcase
end
endmodule

```

Datapath:

```

module acc_pipe(X1, X2, X3, X4, Y, ready, valid, ready_out, valid_out, clk, arst);

    input signed [7:0] X1, X2, X3, X4;
    output signed [7:0] Y;
    input valid, ready_out;
    output ready, valid_out;
    input clk, arst;

    wire signed [7:0] S1, S2;
    wire signed [7:0] Y1, Y2, Y3;
    wire [2:0] enable;

    //parameters for N1
    parameter [7:0] n1_w1 = -8'd115;
    parameter [7:0] n1_w2 = 8'd1;
    parameter [7:0] n1_w3 = -8'd105;

```

```

parameter [7:0] n1_w4 = 8'd16;
parameter [15:0] n1_bias = 16'd12571;
parameter [11:0] n1_xmin = -12'd127;
parameter [11:0] n1_xmax = 12'd127;
//parameters for N2
parameter [7:0] n2_w1 = 8'd103;
parameter [7:0] n2_w2 = -8'd22;
parameter [7:0] n2_w3 = 8'd32;
parameter [7:0] n2_w4 = -8'd56;
parameter [15:0] n2_bias = -16'd8139;
parameter [11:0] n2_xmin = -12'd127;
parameter [11:0] n2_xmax = 12'd127;
//parameters for N3
parameter [7:0] n3_w1 = 8'd75;
parameter [7:0] n3_w2 = -8'd85;
parameter [7:0] n3_w3 = -8'd38;
parameter [7:0] n3_w4 = 8'd92;
parameter [15:0] n3_bias = 16'd10182;
parameter [11:0] n3_xmin = -12'd127;
parameter [11:0] n3_xmax = 12'd127;

//neuroni
neuron Neuron1(X1, X2, X3, X4, n1_w1, n1_w2, n1_w3, n1_w4, n1_bias,
n1_xmin, n1_xmax, Y1);
neuron Neuron2(X1, X2, X3, X4, n2_w1, n2_w2, n2_w3, n2_w4, n2_bias,
n2_xmin, n2_xmax, Y2);
neuron Neuron3(S1, S2, 8'b0, 8'b0, n3_w1, n3_w2, n3_w3, n3_w4,
n3_bias,n3_xmin, n3_xmax, Y3);

//FSM
fsm automa(ready, valid, ready_out, valid_out, clk, arst,enable);

//registri
registro #(8) reg1(clk, arst,enable[0],Y1,S1);
registro #(8) reg2(clk, arst,enable[1],Y2,S2);
registro #(8) reg3(clk, arst,enable[2],Y3,Y);

endmodule

```

Simulazione

È presentato di seguito l'output del test bench fornito, il quale attesta il successo di tutti e 100 i test a cui è stato sottoposto l'acceleratore neurale.

```
100 cases evaluated,          0 errors found
```


Architettura 2

Nella seguente architettura, vengono utilizzati solo 2 neuroni, e ognuno di essi può essere alimentato con gli input dell'acceleratore (X1-X4) oppure con i risultati parziali memorizzati nei due registri interni (Reg 1 e Reg 2). Entrambi i neuroni possono essere utilizzati come N1, N2 o N3 della rete neurale originale.

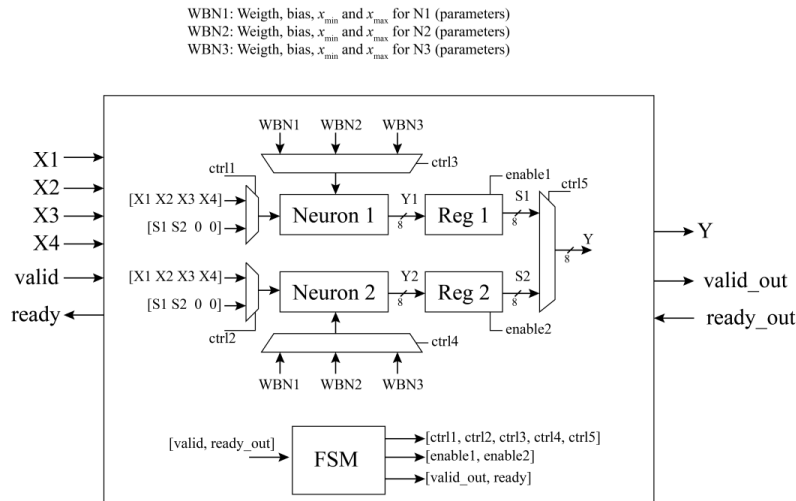


Figure 11: Architettura 2

L'automa deve essere progettato in modo da gestire i segnali dei multiplexer, che alimentano i neuroni con il giusto insieme di valori, garantendo così la corretta esecuzione delle operazioni richieste.

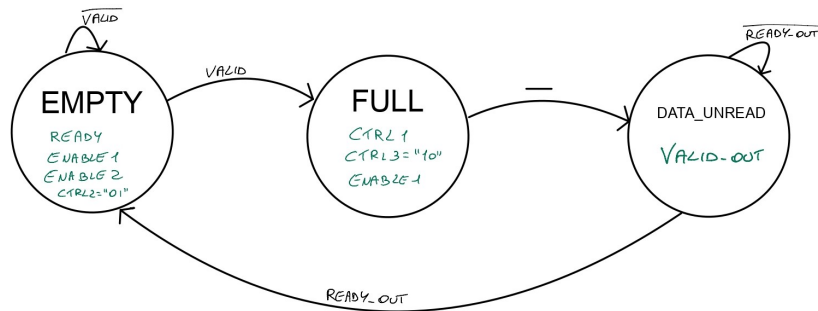


Figure 12: Automa a Stati Finiti 2

La riduzione dei componenti e il riuso degli stessi comportano la perdita di gradi di parallelismo nell'intero sistema. L'automa, quindi, dirige prima i neuroni 1

e 2 per elaborare i valori X1-X4. Successivamente, sarà nuovamente il neurone 1 a prendere il posto del neurone 3 precedente per processare i dati salvati nei due registri. Nello stato finale, l'uscita sarà disponibile nel Reg 1.

Implementazione

Nella descrizione del datapath, la corretta gestione dei segnali in ingresso e in uscita ai vari multiplexer ha avuto un ruolo cruciale. L'operazione di concatenazione e successiva divisione dei segnali è stata fondamentale per garantire il corretto flusso di informazioni all'interno del sistema.

```
module acc_pipe_v2(X1, X2, X3, X4, Y, ready, valid, ready_out, valid_out, clk, arst);

    input signed [7:0] X1, X2, X3, X4;
    output signed [7:0] Y;
    input valid, ready_out;
    output ready, valid_out;
    input clk, arst;

    wire signed [7:0] S1, S2;
    wire signed [7:0] Y1, Y2, Y3;
    wire [1:0] enable;

    //segnali controllo mux
    wire ctrl1,ctrl2,ctrl5;
    wire [1:0] ctrl3,ctrl4;

    //ingressi neuroni
    wire signed [31:0] X;
    assign X={X1,X2,X3,X4};
    wire signed [31:0] S;
    assign S={S1,S2,8'd0,8'd0};

    //uscite multiplexer
    wire signed [31:0] IN1,IN2;
    wire signed [71:0] WBN1,WBN2,WBN3,W1,W2;

    //parameters for N1
    parameter [7:0] n1_w1 = -8'd115;
    parameter [7:0] n1_w2 = 8'd1;
    parameter [7:0] n1_w3 = -8'd105;
    parameter [7:0] n1_w4 = 8'd16;
    parameter [15:0] n1_bias = 16'd12571;
    parameter [11:0] n1_xmin = -12'd127;
    parameter [11:0] n1_xmax = 12'd127;
    assign WBN1={n1_w1,n1_w2,n1_w3,n1_w4,n1_bias,n1_xmin,n1_xmax};
    //parameters for N2
    parameter [7:0] n2_w1 = 8'd103;
    parameter [7:0] n2_w2 = -8'd22;
    parameter [7:0] n2_w3 = 8'd32;
```

```

parameter [7:0] n2_w4 = -8'd56;
parameter [15:0] n2_bias = -16'd8139;
parameter [11:0] n2_xmin = -12'd127;
parameter [11:0] n2_xmax = 12'd127;
assign WBN2={n2_w1,n2_w2,n2_w3,n2_w4,n2_bias,n2_xmin,n2_xmax};
//parameters for N3
parameter [7:0] n3_w1 = 8'd75;
parameter [7:0] n3_w2 = -8'd85;
parameter [7:0] n3_w3 = -8'd38;
parameter [7:0] n3_w4 = 8'd92;
parameter [15:0] n3_bias = 16'd10182;
parameter [11:0] n3_xmin = -12'd127;
parameter [11:0] n3_xmax = 12'd127;
assign WBN3={n3_w1,n3_w2,n3_w3,n3_w4,n3_bias,n3_xmin,n3_xmax};

//neuron1
neuron Neuron1(IN1[31:24], IN1[23:16], IN1[15:8], IN1[7:0],
W1[71:64], W1[63:56], W1[55:48], W1[47:40], W1[39:24], W1[23:12],
W1[11:0], Y1);
neuron Neuron2(IN2[31:24], IN2[23:16], IN2[15:8], IN2[7:0],
W2[71:64], W2[63:56], W2[55:48], W2[47:40], W2[39:24], W2[23:12],
W2[11:0], Y2);

mux_2_1 #(32) MUX1(X,S,ctrl1,IN1);
mux_2_1 #(32) MUX2(X,S,ctrl2,IN2);
mux_2_1 #(8) MUX5(S1,S2,ctrl5,Y);

mux_4_1 #(72) MUX3(WBN1,WBN2,WBN3,72'b0,ctrl3,W1);
mux_4_1 #(72) MUX4(WBN1,WBN2,WBN3,72'b0,ctrl4,W2);

fsm_v2 automa(ready, valid, ready_out, valid_out, clk,
arst,enable,ctrl1,ctrl2,ctrl3,ctrl4,ctrl5);

registro reg1(clk, arst,enable[0],Y1,S1);
registro reg2(clk, arst,enable[1],Y2,S2);

endmodule

Automa a stati finiti:

module fsm_v2(ready, valid, ready_out, valid_out, clk, arst,
en,ctrl1,ctrl2,ctrl3,ctrl4,ctrl5);

input valid, ready_out;
output ready, valid_out;
input clk, arst;
output [1:0] en;
output ctrl1,ctrl2,ctrl5;
output [1:0] ctrl4,ctrl3;

wire clk,arst,valid,ready_out;

```

```

    reg [1:0] en;
    reg ctrl1,ctrl2,ctrl5;
    reg [1:0] ctrl4,ctrl3;
    reg ready=0, valid_out=0;
    reg [2:0] ac_state=3'b000;
    parameter [1:0] EMPTY=3'b000, FULL=3'b001,UNREAD=3'b011;

always @(posedge arst or posedge clk)
begin
    if (arst)
        ac_state <= EMPTY;
    else
        case (ac_state)
            EMPTY: ac_state <= (valid) ? FULL : EMPTY;
            FULL: ac_state <= UNREAD;
            UNREAD: ac_state <= (ready_out) ? EMPTY : UNREAD;
            default: ac_state<=EMPTY;
        endcase
    end

always @(ac_state)
begin
    ctrl1<= 0;
    ctrl2<=0;
    ctrl3<= 0;
    ctrl4<=2'b01;
    ctrl5<=2'b0;
    case (ac_state)
        EMPTY: begin
            ready <= 1;
            valid_out <= 0;
            en = 2'b11;
        end
        FULL: begin
            ready <= 0;
            valid_out <= 0;
            ctrl1<= 1;
            ctrl3<=2'b10;
            en = 2'b01;
        end
        UNREAD: begin
            ready <= 0;
            valid_out <= 1;
            en = 2'b00;
        end
    endcase
end
endmodule

```

Simulazione

È presentato di seguito l'output del test bench fornito, il quale attesta il successo di tutti e 100 i test a cui è stato sottoposto l'acceleratore neurale.

```
100 cases evaluated,          0 errors found
```

Datapath Aritmetico

Design an arithmetic datapath that receives 2 signed 16-bits inputs (A , B) and a 3b-it operation code (opcode) defining the operation computed by the datapath. The outputs of the circuit are the 16-bits signal Y and the single bit signal co . The possible operations, the corresponding opcode and the computed values are shown in the following table:

Operation	Short name	opcode	Computed Y	Computed co
Sum	Sum	3'b000	$A + B$	Carry out
Sum and increment	Sumi	3'b001	$A + B + 1$	Carry out
Subtract and Decrement	Subd	3'b010	$A - B - 1$	Carry out
Subtract	Sub	3'b011	$A - B$	Carry out
Passthrough	Pt1	3'b100	A	Zero
Increment	Inc	3'b101	$A + 1$	Carry out
Decrement	Dec	3'b110	$A - 1$	Carry out
Passthrough	Pt2	3'b111	A	Zero

Figure 13: Operation Table

The following exercises are optional and can be implemented in any order and/or mixed together:

- Use a parameter to define the number of bits for the inputs and outputs of the datapath.
- Add to the testbench the needed code for random generation of the circuit inputs.
- Add to the testbench the needed code for accessing a file containing the circuit inputs.
- Add a parameter 'pipe' to the design. The parameter can have 2 values. If 'pipe' equals 0, the design has no pipeline registers. If 'pipe' equals 1, registers are added to the inputs and to the outputs of the circuit. The datapath shall require another input for the clock signal.
- Implement exercise 2 and consider another possible value for 'pipe'. If 'pipe' equals 2, pipeline registers are added to the inputs, the outputs, and in the middle of the adder. For this exercise, the adder can be either described as a chain of full-adders (as in the carry-ripple architecture), or the sum operation can be divided into two statements working on the LSBs and MSBs of A and B respectively. For the carry-ripple architecture, the Verilog description of the full-adder must be added to the hierarchy.

Progetto e Architettura

L'architettura richiesta dalla specifica è la seguente:

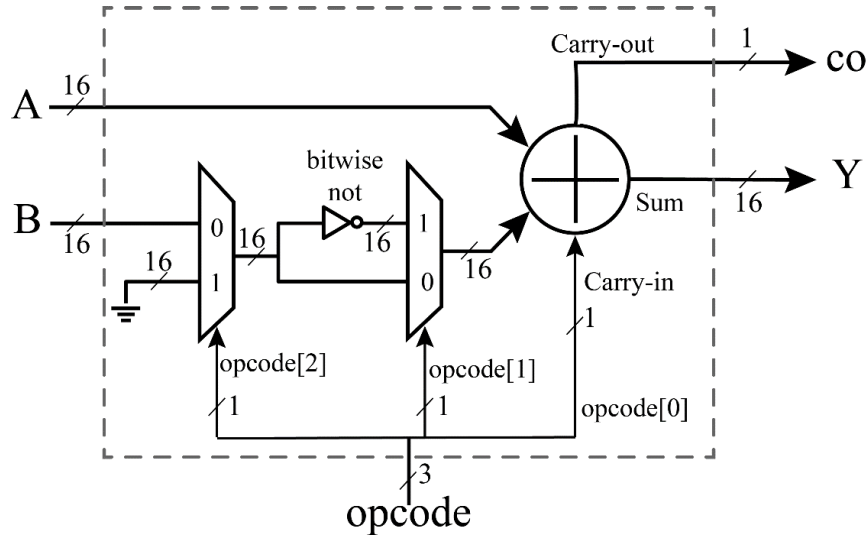


Figure 14: Datapath

Il datapath implementa le otto operazioni utilizzando un sommatore, due multiplexer e 16 porte NOT. Il bit meno significativo del codice operativo viene utilizzato come carry-in per il sommatore. Gli altri due bit del codice operativo modificano il secondo ingresso del sommatore, invertendo o azzerando i suoi bit a seconda delle operazioni richieste.

Datapath Pipelined

Nel seguente paragrafo sono mostrate le due implementazioni pipelined.

Add a parameter 'pipe' to the design. The parameter can have 2 values. If 'pipe' equals 0, the design has no pipeline registers. If 'pipe' equals 1, registers are added to the inputs and to the outputs of the circuit. The datapath shall require another input for the clock signal.

La seguente richiesta è facilmente implementata inserendo 4 registri:

- 2 registri per gli ingressi A e B
- 2 registri per le uscite Y e Co

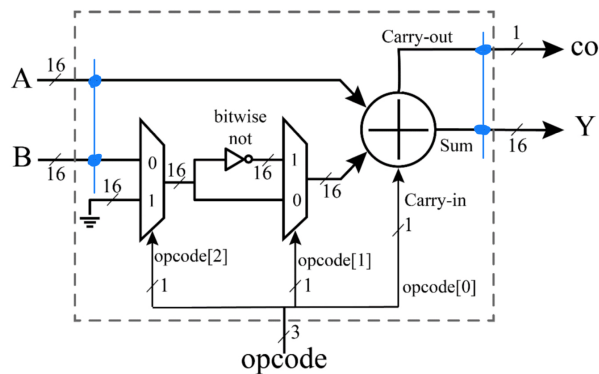


Figure 15: Datapath Pipelined 1

If 'pipe' equals 2, pipeline registers are added to the inputs, the outputs, and in the middle of the adder. For this exercise, the adder can be either described as a chain of full-adders (as in the carry-ripple architecture), or the sum operation can be divided into two statements working on the LSBs and MSBs of A and B respectively. For the carry-ripple architecture, the Verilog description of the full-adder must be added to the hierarchy.

Per questa implementazione, si è scelto di rendere l'architettura del ripple carry adder pipelined. È stato inserito un registro nella catena del riporto del carry, dividendo così la catena a metà.

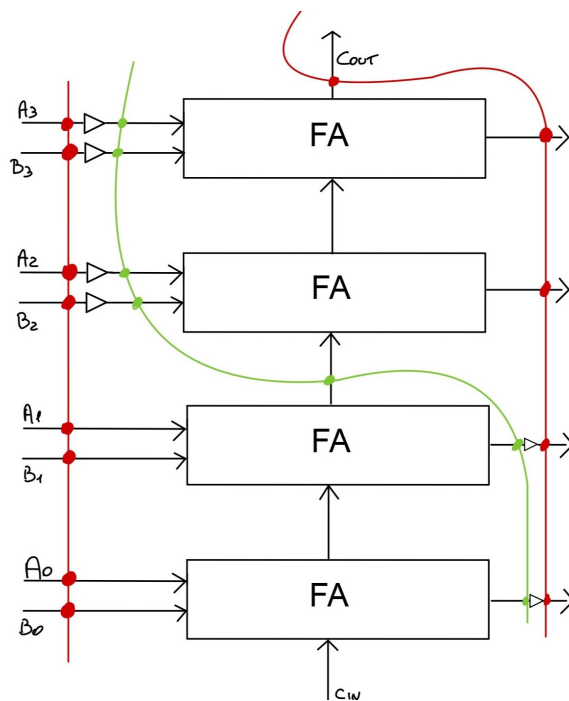


Figure 16: RCA a 4 bit con una pipe a due livelli

Nell'implementazione del Ripple Carry Adder, l'architettura è modellata come un grafo, dove gli ingressi e le uscite sono disposti ai lati opposti. Un registro è aggiunto ogni volta che un arco attraversa il grafo, spezzandolo. Per rispettare le specifiche, inizialmente, è stato inserito un livello di pipeline sia sulle uscite che sugli ingressi. Successivamente, è stata eseguita una divisione a metà dell'addizionatore, inserendo un registro al centro della catena di carry. Ciò ha comportato l'aggiunta di registri sia sugli ingressi che sulle uscite. Per evitare situazioni critiche relative al vincolo di hold, è stato inserito un buffer tra due registri in serie.

***Nota:** Va sottolineato che l'aggiunta di un singolo registro solo nella catena del riporto, senza l'accompagnamento di registri corrispondenti sugli ingressi e sulle uscite, avrebbe compromesso il corretto funzionamento del circuito. Questa scelta avrebbe infatti compromesso la corretta temporizzazione dei segnali rispetto al segnale di abilitazione dei registri (clock).*

Implementazione

I componenti non presentati in questa sezione sono discussi in APPENDICE.

Rippe Carry Adder

Si mostra di seguito il codice del Ripple Carry Adder su N bit , dove N è parametrico.

```
module RippleCarryAdder
  #(parameter N=16)
  (input wire [N-1:0] A, B,
   input wire Cin,
   output wire [N-1:0] Sum,
   output wire Cout
  );

  wire [N-1:0] Carry;

  //Primo Full adder
  FullAdder FA (
    .A(A[0]),
    .B(B[0]),
    .Cin(Cin),
    .Sum(Sum[0]),
    .Cout(Carry[0])
  );
  // Full Adder per ogni bit
  genvar i;
  generate
    for (i = 1; i < N; i=i+1) begin: gen_adder
      FullAdder FA (
        .A(A[i]),
        .B(B[i]),
        .Cin(Carry[i-1]),
```

```

        .Sum(Sum[i]),
        .Cout(Carry[i])
    );
end
endgenerate

assign Cout = Carry[N-1];

endmodule

```

Di conseguenza si mostra anche il codice del singolo Full adder.

```

module FullAdder
    (input wire A, B, Cin,
     output wire Sum, Cout);

    //assign Sum = A ^ B ^ Cin; //A xor B xor Cin

    //In tecnologica Cmos è vantaggioso scriverlo in termini di Cout
    assign Sum = (~ Cout & (A | B | Cin)) | (A & B & Cin);
    assign Cout = (A & B) | (B & Cin) | (Cin & A);
endmodule

```

Di seguito, invece, viene mostrato anche un approccio comportamentale alla somma:

```

module sum
    #(parameter N=16)
    (input signed [N-1:0] A, B,
     input opcode,
     output signed [N-1:0] Y,
     output c0
    );
    wire [1:0] temp;

    assign temp = {1'b0, opcode};
    assign {c0, Y} = A + B + temp;

endmodule

```

Buffer

Il buffer è inserito tra due registri disposti in serie al fine di evitare di rendere critico il vincolo di hold.

```

module buffer #(parameter N=16) (
    input signed [N-1:0] A,
    output signed [N-1:0] Y
);

assign Y=A;

```

```
endmodule
```

Datapath

Si presenta infine l'architettura del sistema complessivo

```
module datapath #(parameter pipe = 2,N=16) (
    input signed [N-1:0] A, B,
    input [2:0] opcode,
    input wire enable_A,
    input wire enable_B,
    input wire enable_out,
    input wire clock,
    input wire reset,
    output signed [N-1:0] Y,
    output wire co
);

    //0-vector
    wire signed [N-1:0] op2;
    assign op2 = {(N){1'b0}};

    //wire mux
    wire signed [N-1:0] M1, M2;

    //wire per registri ingresso/uscite
    wire signed [N-1:0] A_r, B_r, Y_r,M2_r,S_r;
    wire C_r;

    //wire per registri all'interno del RCA
    wire signed [N/2-1:0] A_temp,M2_temp,S_temp;
    wire C_temp,Ctemp_out;

    //wire buffer
    wire signed [N/2-1:0] S_buffer,A_buffer,M2_Buffer;

    generate
        if (pipe == 0)
        begin : pipe0
            mux_2_1 mux1(B, op2 , opcode[2], M1);
            mux_2_1 mux2(M1, ~M1, opcode[1], M2);
            sum sum1(A, M2, opcode[0], Y, co);
        end

        else if (pipe == 1)
        begin : pipe1
            mux_2_1 mux1(B_r, op2, opcode[2], M1);
            mux_2_1 mux2(M1, ~M1, opcode[1], M2);
            //sommatore
            RippleCarryAdder #(N)summer(A_r,M2,opcode[0],Y_r,C_r);
```

```

        //registri su ingressi
        registro #(N)regA(clock, reset, enable_A, A, A_r);
        registro #(N)regB(clock, reset, enable_B, B, B_r);
        //registri sulle uscite
        registro #(N)regY(clock, reset, enable_out, Y_r , Y);
        registro #(1) regC(clock, reset, enable_out, C_r , co);
    end

else if(pipe == 2)
    begin: pipe2
        //mux
        mux_2_1 mux1(B_r, op2, opcode[2], M1);
        mux_2_1 mux2(M1, ~M1, opcode[1], M2);

        //registro su B che ha un percorso combinatorio prima del sommatore
        registro #(N)regB(clock, reset, enable_B, B, B_r);

        //prima pipe --archi rossi
        registro #(N)regA(clock, reset, enable_A, A, A_r);
        registro #(N)regM2(clock, reset, enable_A, M2, M2_r);
        registro #(N)regY(clock, reset, enable_out, S_r , Y);
        registro #(1) regC(clock, reset, enable_out, C_r , co);

        //seconda pipe --arco verde
        buffer #(N/2) buffer_A(A_r[N-1:N/2],A_buffer);
        registro #(N/2)regA_temp(clock, reset, 1'b1, A_buffer,A_temp);
        buffer #(N/2) buffer_M2(M2_r[N-1:N/2],M2_Buffer);
        registro #(N/2)regM_temp(clock, reset, 1'b1,M2_Buffer, M2_temp);
        //uscita
        registro #(N/2)regS_temp(clock, reset, 1'b1, Y_r[N/2-1:0], S_temp);
        buffer #(N/2) buffer_S(S_temp,S_buffer);

        //RCA Pipelined a due livelli
        RippleCarryAdder #(N/2)rca_1(A_r[N/2-1:0],
            M2_r[N/2-1:0],opcode[0],Y_r[N/2-1:0],C_temp);
        //registro nella catena del riporto
        registro #(1) regC_tmp(clock, reset, 1'b1, C_temp , Ctemp_out);
        RippleCarryAdder #(N/2)rca_2(A_temp,M2_temp,Ctemp_out,
            Y_r[N-1:N/2],C_r);

        //somma in ingresso all'ultimo livello di pipe uscite
        //uscita buffer primo stadio + uscita secondo RCA
        assign S_r ={Y_r[N-1:N/2],S_buffer};

    end
endgenerate

endmodule

```

Simulazione

Il test bench fornito è stato opportunamente modificato per implementare ulteriori funzioni richieste nella specifica.

Generazione casuale input

La funzione random è la responsabile di generare i codici in maniera randomica:

```
$display("Start random generation");
for (int i = 0; i < 10; i = i + 1) begin
    $random(opcode); A={$random} % 100; B={$random} % 100;
    #10;
    $display("[time: %0dns, %0s] OP:%0b A:%0d, B:%0d, Y:%0d,
    co:%b",$time,operation(opcode), opcode,A, B, Y, co);
end
```

Lettura da file

Di seguito il codice per la lettura da file degli input:

```
integer file;
$display("Start reading from file");
file= $fopen("input.txt","r");
if(file==0)
    begin
        $display("File non trovato");
        $finish;
    end
while(!$feof(file))
    begin

        $fscanf (file,"%b %d %d\n", opcode,A, B);
        #10;
        $display("[time: %0dns, %0s] OP:%0b A:%0d, B:%0d, Y:%0d,
        co:%b",$time, operation(opcode),opcode,A, B, Y, co);

    end
```

Riconoscimento codice operazione

E' stata infine aggiunta una funzione che riconosca l'operazione eseguita in base al codice operativo OP:

```
function string operation(reg [2:0] opcode);
case(opcode)
    3'b000: operation = "Sum";
    3'b001: operation = "Sumi";
    3'b010: operation = "Subd";
    3'b011: operation = "Sub";
    3'b100: operation = "Pt1";
    3'b101: operation = "Inc";
```

```

        3'b110: operation = "Dec";
        3'b111: operation = "Pt2";
        default: operation = "Null";
    endcase
endfunction

```

Test Bench

Si mostra il codice completo:

```

`define maxpositive 32767
`define minnegative -32768

`include "datapath.v"
`timescale 1ns/1ps

module datapathTB();
    reg signed [15:0] A,B;
    reg [2:0] opcode;
    wire signed [15:0] Y;
    wire co;
    integer file;
    reg clk = 1'b0;
    reg arst;

    datapath #(.pipe(2)) myDatapath (.A(A), .B(B), .opcode(opcode), .Y(Y),
    .co(co), .clock(clk), .reset(arst), .enable_A(1'b1), .enable_B(1'b1), .enable_out(1'b1));

    function string operation(reg [2:0] opcode);
        case(opcode)
            3'b000: operation = "Sum";
            3'b001: operation = "Sumi";
            3'b010: operation = "Subd";
            3'b011: operation = "Sub";
            3'b100: operation = "Pt1";
            3'b101: operation = "Inc";
            3'b110: operation = "Dec";
            3'b111: operation = "Pt2";
            default: operation = "Null";
        endcase
    endfunction

    always
        begin
            #2;
            clk <= ~clk;
        end
    initial
        begin : initLabel
            arst=1;

```

```

#10
arst=0;
// operation: Sum
opcode = 3'b000; A = 16'd5; B = 16'd128;
#30;
$display("[time: %0dns, sum] A:%0d, B:%0d, Y:%0d, co:%b",$time, A, B, Y, co);

// operation: Sumi
opcode = 3'b001; A = 16'd5; B = 16'd128;
#20;
$display("[time: %0dns, sumi] A:%0d, B:%0d, Y:%0d, co:%b",$time, A, B, Y, co);

// operation: Subd
opcode = 3'b010; B = 16'd5; A = 16'd10;
#20;
$display("[time: %0dns, subd] A:%0d, B:%0d, Y:%0d, co:%b",$time, A, B, Y, co);

// operation: Sub
opcode = 3'b011; A = 16'd5; B = 16'd128;
#20;
$display("[time: %0dns, sub] A:%0d, B:%0d, Y:%0d, co:%b",$time, A, B, Y, co);

// random generate
$display("Start random generation");
for (int i = 0; i < 10; i = i + 1) begin
    $random(opcode); A={$random} % 100; B={$random} % 100;
    #20;
    $display("[time: %0dns, %0s] OP:%0b A:%0d, B:%0d, Y:%0d,
    co:%b",$time,operation(opcode), opcode,A, B, Y, co);
end

$display("Start reading from file");
file= $fopen("input.txt","r");
if(file==0)
    begin
        $display("File non trovato");
        $finish;
    end
while(!$feof(file))
    begin
        $fscanf (file,"%b %d %d\n", opcode,A, B);
        #20;
        $display("[time: %0dns, %0s] OP:%0b A:%0d, B:%0d, Y:%0d,
        co:%b",$time, operation(opcode),opcode,A, B, Y, co);
    end

$finish;
end
endmodule

RISULTATI PIPE=2:

```

```

# KERNEL: [time: 40ns, sum] A:5, B:128, Y:133, co:0
# KERNEL: [time: 60ns, sumi] A:5, B:128, Y:134, co:0
# KERNEL: [time: 80ns, subd] A:10, B:5, Y:4, co:1
# KERNEL: [time: 100ns, sub] A:5, B:128, Y:-123, co:0
# KERNEL: Start random generation
# KERNEL: [time: 120ns, Sum] OP:0 A:48, B:97, Y:145, co:0
# KERNEL: [time: 140ns, Dec] OP:110 A:57, B:87, Y:56, co:1
# KERNEL: [time: 160ns, Pt2] OP:111 A:57, B:57, Y:57, co:1
# KERNEL: [time: 180ns, Pt1] OP:100 A:25, B:82, Y:25, co:0
# KERNEL: [time: 200ns, Inc] OP:101 A:61, B:29, Y:62, co:0
# KERNEL: [time: 220ns, Subd] OP:10 A:18, B:97, Y:-80, co:0
# KERNEL: [time: 240ns, Sub] OP:11 A:29, B:12, Y:17, co:1
# KERNEL: [time: 260ns, Sum] OP:0 A:89, B:26, Y:115, co:0
# KERNEL: [time: 280ns, Dec] OP:110 A:97, B:22, Y:96, co:1
# KERNEL: [time: 300ns, Pt2] OP:111 A:85, B:55, Y:85, co:1
# KERNEL: Start reading from file
# KERNEL: [time: 320ns, Dec] OP:110 A:97, B:22, Y:96, co:1
# KERNEL: [time: 340ns, Inc] OP:101 A:59, B:106, Y:60, co:0
# KERNEL: [time: 360ns, Pt1] OP:100 A:78, B:15, Y:78, co:0
# KERNEL: [time: 380ns, Dec] OP:110 A:41, B:64, Y:40, co:1
# KERNEL: [time: 400ns, Pt2] OP:111 A:82, B:33, Y:82, co:1
# KERNEL: [time: 420ns, Dec] OP:110 A:15, B:92, Y:14, co:1
# KERNEL: [time: 440ns, Inc] OP:101 A:41, B:64, Y:42, co:0
# KERNEL: [time: 460ns, Dec] OP:110 A:110, B:97, Y:109, co:1
# KERNEL: [time: 480ns, Inc] OP:101 A:82, B:59, Y:83, co:0
# KERNEL: [time: 500ns, Pt1] OP:100 A:33, B:78, Y:33, co:0

```


Appendice

Multiplexer

Il seguente codice descrive un multiplex 2:1 parametrico rispetto al numero di bit in ingresso N.

```
module mux_2_1 #(parameter N=16)(
    input signed [N-1:0] A, B,
    input select,
    output signed [N-1:0] Y
);
    assign Y = select ? B : A;

endmodule
```

Il seguente codice descrive un multiplex 4:1 parametrico rispetto al numero di bit in ingresso N.

```
module mux_4_1 #(parameter N=8)(
    input signed [N-1:0] A, B,C,D,
    input [1:0] select,
    output signed [N-1:0] Y
);
    assign Y = select[0] ?
        (select[1] ? D : B) :
        (select[1] ? C : A);

endmodule
```

Registro

Di seguito il codice di un registro in grado di memorizzare M bit.

```
module registro #(parameter M=16) (
    input clock,
    input arst,
    input enable,
    input [M-1:0] D,
    output [M-1:0] Y
);
    reg [M-1:0] Q;
    always @(posedge clock or posedge arst) begin
        if (arst)
            Q <= {(M){1'b0}};
        else if (clock)
            if (enable)
                Q <= D;
    end
    assign Y = Q;

endmodule
```